

Identity & Access Management (IAM)

API Prompt

I want to build a complete backend API for an Identity & Access Management (IAM) module for a Travel Management System using a microservices-friendly architecture.

TECHNOLOGY STACK

- REST API
- Prefer Spring Boot (Java) OR Node.js (Express)
- Layered architecture (Controller, Service, Repository)
- Use DTOs, validation, exception handling
- Implement JWT-based authentication
- Use role-based access control (RBAC)
- Follow clean code and scalable design

MODULE: IDENTITY & ACCESS MANAGEMENT (IAM)

This module handles:

- User registration and login
- Role-based access control (RBAC)
- Audit logging of all user actions

- Grade-based classification (for travel policies)

ENTITY: USER

Fields:

- UserID (Primary Key)
- Name
- Role (Enum: Employee, TravelDesk, ApprovingManager, Finance, Compliance, Admin)
- Email (Unique, used for login)
- Password (hashed securely using bcrypt or similar)
- Phone
- GradeID (FK → Grade, used for travel policy mapping)
- DepartmentID
- Status (Active / Inactive)
- CreatedDate
- UpdatedDate

ENTITY: GRADE

Fields:

- GradeID (Primary Key, e.g., G1, G2, G3, G4, G5, G6)

- GradeName (e.g., Junior Employee, Senior Employee, Manager, Senior Manager, Director, Executive)
- Description (optional)
- CreatedDate
- UpdatedDate
- Status (Active / Inactive)

PREDEFINED GRADES (DEFAULT DATA)

System must initialize with:

- G1 → Junior Employee
- G2 → Senior Employee
- G3 → Manager
- G4 → Senior Manager
- G5 → Director
- G6 → Executive / VP

ENTITY: AUDIT LOG

Fields:

- AuditID (Primary Key)

- UserID (FK → User)
- Action (e.g., LOGIN, REGISTER, CREATE_USER, UPDATE_USER, APPROVE_REQUEST, UPDATE_POLICY)
- Module (e.g., IAM, TravelPolicy, CityTier, Booking)
- Timestamp
- Details (optional description)

REQUIREMENTS

1. AUTHENTICATION APIs:

- Register User → POST /auth/register
- Login User → POST /auth/login
- Logout (optional)
- Change Password
- Reset Password (optional)

Login must:

- Validate email & password
- Return JWT token
- Token must include UserID, Role, GradeID
- Block login if Status = Inactive

2. USER MANAGEMENT APIs:

- Create User (Admin only)

- Update User
- Get User by ID
- Get All Users
- Deactivate User

3. GRADE MANAGEMENT APIs:

- Create Grade (Admin only)
- Update Grade
- Get all Grades
- Get Grade by ID
- Deactivate Grade

Rules:

- GradeID must be unique
- GradeName cannot be empty
- Grades reusable across modules

4. ROLE-BASED ACCESS CONTROL (RBAC):

- Employee → create request
- ApprovingManager → approve/reject
- TravelDesk → bookings
- Finance → reimbursements
- Compliance → audit
- Admin → full access

System must restrict APIs based on role using JWT.

5. AUDIT LOGGING:

Log actions like LOGIN, REGISTER, CREATE_USER, UPDATE_USER, DELETE_USER.

Store:

- UserID
- Action
- Module
- Timestamp

Provide filtering APIs.

6. VALIDATION:

- Email unique
- Password strong
- Role valid
- Phone format valid
- Status Active/Inactive
- GradeID must exist

7. SECURITY:

- Password hashing (bcrypt)
- JWT authentication
- Protected endpoints

8. ERROR HANDLING:

Use proper HTTP status codes and consistent JSON errors.

9. DATABASE:

Tables: User, Grade, AuditLog

Constraints:

- Email unique
- Foreign keys

10. FILTERING:

- By role
- By grade
- Active users
- Search by name/email
- Audit filters

IMPORTANT:

Role = access

Grade = benefits

OUTPUT EXPECTED

Provide:

1. Entity classes

2. DTOs
3. Repository
4. Service
5. Controller
6. JWT implementation
7. RBAC logic
8. Audit logging
9. Validation
10. Exception handling
11. SQL schema
12. Sample API JSON

Ensure production-ready code and no missing features.